# HPDP Process Description

How the project team made the Hyperspectral Payload Data Processor, from the specification to the tests on the board

| | |
|---|---|
| **Document** | HPDP-PRC-001, revision B |
| **Date** | 2026-10-23 |
| **Prepared by** | Shubham Draveriya |
| **System** | Hyperspectral Payload Data Processor (HPDP) on the AMD ZCU106 board |
| **Language** | ASD-STE100 Simplified Technical English (STE) |
| **Related documents** | SPEC.md (specification v1.0), HPDP-BRF-001 (project brief), HPDP-RPT-001 (results report), HPDP-GDE-001 (learning guide), docs/m0 to m7 (milestone notes) |

This document tells how the project team made the HPDP, from the first specification to the tests on the board. It also gives the procedures to build, test and operate the system again.

# Contents

**1 About this document . . . 3**
1.1 Purpose . . . 3
1.2 Who can use this document . . . 3
1.3 Writing rules . . . 3
1.4 Warnings, cautions and notes . . . 3
**2 The HPDP and its purpose . . . 3**
2.1 What the HPDP is . . . 3
2.2 Why a satellite needs this data chain . . . 4
2.3 The reference mission . . . 4
**3 The data chain . . . 5**
3.1 Overview . . . 5
3.2 The satellite part . . . 5
3.3 The ground part . . . 6
3.4 Control and software . . . 6
**4 The method of work . . . 7**
4.1 Principles . . . 7
4.2 Tools and equipment . . . 7
**5 The project phases . . . 8**
5.1 The start: from an image project to a satellite project . . . 8
5.2 M0: Tools and board start-up . . . 8
5.3 M1: Python models . . . 9
5.4 M2: The compressor in hardware . . . 9
5.5 M3: The sensor front end . . . 10
5.6 M4: First compression on the board . . . 10
5.7 M5: Downlink chain and optical link . . . 10
5.8 M6: The PC ground station . . . 11
5.9 M7: Radiation mitigation . . . 11
5.10 After M7 . . . 12
**6 Verification . . . 13**
**7 Procedures . . . 14**
7.1 Prepare the PC . . . 14
7.2 Do the model tests . . . 14
7.3 Do the RTL simulation tests . . . 14
7.4 Build the FPGA configuration . . . 14
7.5 Build the software . . . 15
7.6 Connect the board . . . 15
7.7 Do a board run . . . 15
7.8 Decode the data at the ground station . . . 15
7.9 Troubleshooting . . . 16
**8 Lessons learned . . . 17**
**9 Status . . . 17**
**10 Technical names and abbreviations . . . 18**

# 1 About this document

## 1.1 Purpose

This document tells how the project team made the HPDP. The HPDP is the Hyperspectral Payload Data Processor. The document gives the sequence of the work, the methods, the tests and the results of each phase. It also gives the procedures that you must do to build and operate the system again.

## 1.2 Who can use this document

Use this document if you continue the project or if you review it. You can also use it to learn how the data chain of an observation satellite operates. You must know the basic terms of digital electronics. You do not have to know the space data standards before you start. Section 10 gives the technical names and abbreviations.

## 1.3 Writing rules

This document uses ASD-STE100 Simplified Technical English (STE). STE is a controlled language for technical documents. These are the most important rules in this document:

- A sentence in a procedure has a maximum of 20 words.
- A sentence in a description has a maximum of 25 words.
- Each step of a procedure gives one instruction.
- Instructions use the imperative form of the verb, for example “Connect the cable”.
- Each word has only one meaning.
- Technical names, for example “Reed-Solomon encoder”, are used where necessary. Section 10 tells what they mean.

## 1.4 Warnings, cautions and notes

| Type | Meaning |
|---|---|
| **WARNING** | Tells you about a risk of injury to persons. |
| **CAUTION** | Tells you about a risk of damage to the equipment, to the data or to the test results. |
| **NOTE** | Gives information that helps you. |

# 2 The HPDP and its purpose

## 2.1 What the HPDP is

The HPDP is a hardware model of the data processor in a hyperspectral Earth observation satellite. A hyperspectral camera records each point of the ground in many narrow colour bands. The HPDP receives the raw camera data and prepares it for transmission to the ground. A ground receiver and a PC program then get the original image back, with no change to a single bit.

The HPDP operates on one AMD ZCU106 evaluation board. The board has a Zynq UltraScale+ device (XCZU7EV). This device has two parts:

- The processing system (PS). This part has four Arm processors. One processor runs the control software.
- The programmable logic (PL). This part contains the data chain. The data chain is digital logic that the team wrote in SystemVerilog.

An optical fiber connects two SFP optical modules on the same board. One module is the transmitter of the satellite. The other module is the receiver of the ground station.

## 2.2 Why a satellite needs this data chain

The data chain must solve four problems:

1. The camera makes more data than the radio link can send. The HPDP compresses the data without loss of information (lossless compression).
2. The radio link adds bit errors. The HPDP adds error correction codes, and the ground station uses them to correct the errors.
3. The ground station must find the start of each data block in a continuous bit stream. The HPDP adds sync markers and frame headers.
4. Radiation in space can change bits in the FPGA. The HPDP finds and corrects these changes, or it marks the data that has errors.

## 2.3 The reference mission

The project uses a scaled reference mission. Table 1 gives its data.

*Table 1: The reference mission*

| Parameter | Value |
|---|---|
| Pixels across the ground track (Nx) | 512 |
| Spectral bands (Nz) | 150 |
| Bits for each sample (D) | 12, in 16-bit containers |
| Line rate | 1000 lines each second |
| Raw sample rate | 76.8 million samples each second (76.8 Msamples/s) |
| Raw data rate | 921.6 Mbit/s |
| Optical link | 1.25 Gbaud; approximately 857 Mbit/s for data packets |
| Scene length (Ny) | 256 lines |

The raw data rate (921.6 Mbit/s) is more than the capacity of the link (857 Mbit/s). Because of this, compression is necessary.

# 3 The data chain

## 3.1 Overview

Figure 1 shows the data chain. The data goes through the blocks in the sequence of the arrows. Each block in the PL processes one sample or one byte in each clock cycle.

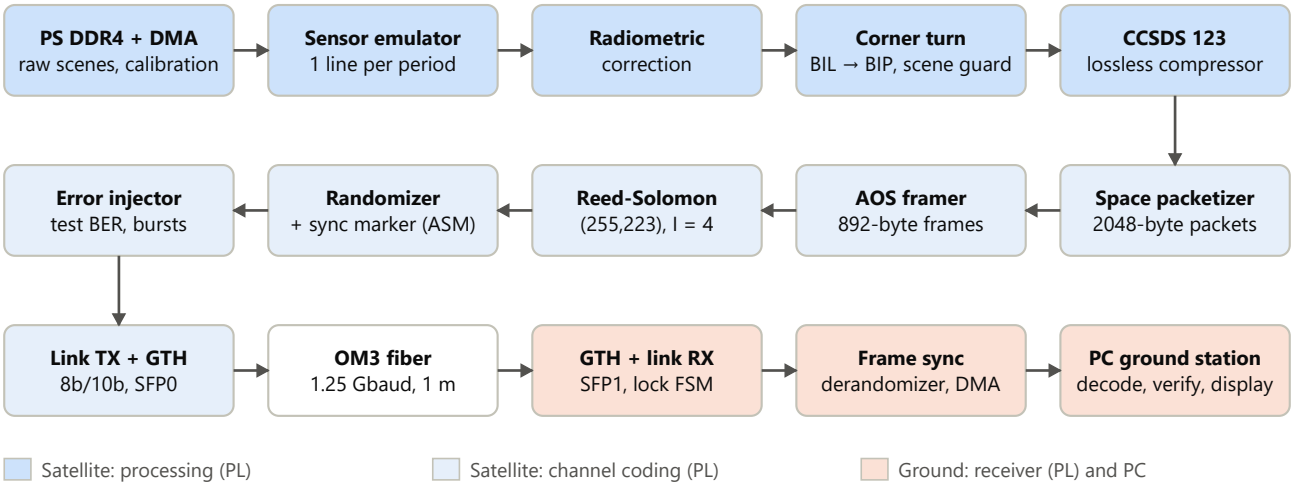



*Figure 1: The HPDP data chain. The satellite part and the ground part are on the same board, connected by the optical fiber.*

## 3.2 The satellite part

1 **Sensor emulator.** This block replaces the real camera. It reads raw scenes from the DDR memory through a DMA controller. It sends one line of samples at a fixed interval, as a real pushbroom camera does. It cannot wait for the blocks after it. If a block after it is too slow, a counter records the lost data.

2 **Radiometric correction.** This block removes the errors of each detector pixel. It subtracts a dark offset and multiplies by a gain. It replaces the value of a bad pixel with the average of its two neighbours. A calibration table in UltraRAM holds these values for each pixel and band.

3 **Corner turn.** The camera sends the samples band by band (BIL order). The compressor needs them pixel by pixel (BIP order). The corner turn writes each line into a buffer and reads it in the other order. It has two buffers, so it can write one line while it reads the line before it.

4 **Scene guard.** This block makes sure that each scene has the correct number of lines. If a line is lost, the scene guard adds a line of fill data. Because of this, one lost line cannot mix two scenes.

5 **CCSDS 123 compressor.** The compressor uses the CCSDS 123.0-B-2 standard for lossless compression. It calculates a prediction for each sample from the samples near it, in the same band and in the previous bands. It codes only the difference between the sample and its prediction. A good prediction gives a small difference, and a small difference needs few bits. The compressor processes one sample in each clock cycle at 250 MHz.

6 **Space packetizer.** The packetizer divides each compressed scene into CCSDS space packets of 2048 bytes. Each packet has a header with the scene number, the packet number and the time of the scene.

7 **AOS framer.** The framer puts the packets into transfer frames of 892 bytes (CCSDS AOS standard). A packet can continue from one frame into the next frame. When no data is ready, the framer sends idle frames. Because of this, the link never stops.

8 **Reed-Solomon encoder.** The encoder adds 128 check bytes to each frame. The result is a codeblock of 1020 bytes. The codeblock contains four interleaved codewords. The ground station can correct up to 16 wrong bytes in each codeword.

9 **Randomizer and sync marker.** The randomizer mixes the bits with a fixed pseudo-random sequence. This gives the receiver enough bit transitions to recover the clock. A 4-byte attached sync marker (ASM) goes before each codeblock. The result is a CADU of 1024 bytes.

10 **Error injector.** This block adds bit errors on purpose. It replaces the noise of a real radio channel during the tests. It can add random errors at a set bit error rate (BER), bursts of errors, or errors in the sync markers.

11 **Link layer and transmitter.** The link layer sends the bytes to a GTH transceiver. The transceiver uses 8b/10b coding. It sends 1.25 Gbaud through the SFP0 module into the optical fiber. When no data is ready, the link layer sends idle words.

## 3.3 The ground part

12 **Link receiver.** The SFP1 module and a second GTH transceiver receive the signal. The link layer finds the word boundaries. It goes into the LOCKED state after 1024 correct words in sequence.

13 **Frame synchronizer.** This block finds the sync markers in the byte stream. It accepts a marker that has up to two wrong bits. It removes the randomization. It writes each codeblock and a status word into the DDR memory through a DMA controller.

14 **PC ground station.** A Python program on the PC reads these records. It corrects the errors with the Reed-Solomon code and takes the packets out of the frames. It puts each scene together, decompresses it, compares it with the reference and shows the image.

## 3.4 Control and software

The control software (hpdp_run) runs on one Arm processor in the PS. It sets the registers of the blocks, starts the DMA transfers and reads the counters. A PC script loads the FPGA configuration, the software, the scenes and a job description through the JTAG cable. The software does the job and writes a result block. The PC script then reads the result block.

One SystemRDL file describes all the registers. A tool (PeakRDL) makes the register logic, the C header and the documentation from this file. Because of this, the hardware and the software always agree.

The PL uses five clocks. The data path uses 250 MHz. The sensor emulator and the registers use 100 MHz. The link uses 62.5 MHz. The transceiver control and the radiation monitor use 31.25 MHz. Special circuits move the data safely from one clock to another (clock domain crossings).

# 4 The method of work

## 4.1 Principles

The team used these six principles in all the phases:

1 **Write the specification first.** The specification (SPEC.md) gives each requirement an identifier, for example CMP-03. Each test and each result refers to these identifiers.

2 **Make a software model first.** For each block, the team wrote a bit-exact model in Python before the hardware. “Bit-exact” means that the model gives the same bits as the hardware.

3 **Test the hardware against the model.** The model makes the test data and the expected results. Each testbench compares every output bit with the expected result. The testbench also stops the data flow at random times.

4 **Work in small steps.** The project has eight milestones (M0 to M7). Each milestone has a clear condition that tells when it is complete.

5 **Test on the board only through JTAG.** The FPGA configuration and the software go into memory that loses its contents at power-off. The team did not change the flash memory of the board.

6 **Record everything.** Each step has a Git commit and a written note in the docs folder.

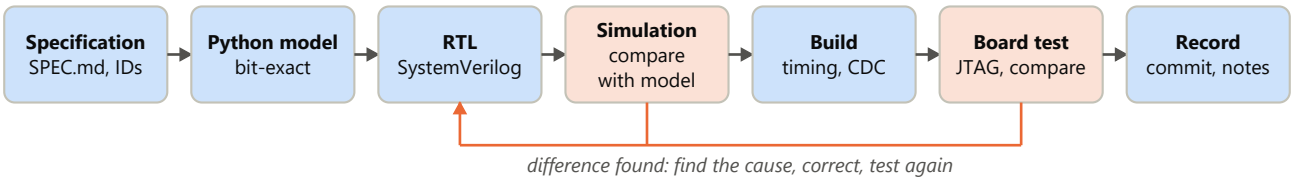



*Figure 2: The work method for each block and each milestone.*

When a test finds a difference, the team finds the cause first. Then the team corrects the hardware, the model or the test, and does the test again. A test passes only when all the bits agree.

## 4.2 Tools and equipment

| Tool or equipment | Use |
|---|---|
| AMD Vivado 2025.2 | Logic synthesis, implementation, timing analysis and simulation (xsim) |
| AMD Vitis 2025.2 | Software build for the Arm processor; XSDB for the access through JTAG |
| Python 3 with pytest | Models, test data generators, PC scripts and the ground station |
| PeakRDL | Register logic, C header and documentation from the SystemRDL file |
| Git | Version control (51 commits from 10 to 15 October 2026) |
| AMD ZCU106 board | The target hardware (XCZU7EV-2 device, DDR4 memory, two SFP cages) |
| Two 1000BASE-SX SFP modules, 1 m OM3 duplex fiber | The optical link |
| PC with two USB connections | JTAG and UART (serial console) to the board |

# 5 The project phases

Table 2 gives the phases, their dates and their results. The subsections after the table give the details of each phase.

*Table 2: The project phases*

| Phase | Date (2026) | Complete when | Result |
|---|---|---|---|
| Start | 10 Oct | Specification written | SPEC.md v1.0 with 102 requirements |
| M0 | 10 Oct | DMA loop test passes on the board | 64 MB with no error at 3865 MB/s |
| M1 | 10 Oct | Python models complete | 138 automatic tests pass (150 at the end) |
| M2 | 10 Oct | Compressor bit-exact, 200 MHz or more | 28 configurations bit-exact, 250 MHz |
| M3 | 10 Oct | Front end agrees with the models | 12 tests bit-exact, 250 MHz |
| M4 | 12 Oct | Board output bit-exact; rate measured | 15 scenes bit-exact, 99.8 Msamples/s |
| M5 | 12 Oct | Link and end-to-end tests pass | $3.4 \times 10^{12}$ bits with no error; scenes bit-exact up to BER $10^{-3}$ |
| M6 | 12 to 15 Oct | Ground station decodes and shows the data | Complete; Ethernet upload tested on the board on 23 Oct |
| M7 | 13 Oct | Radiation mitigation complete | Upsets corrected in 153 ms or less; no silent error in protected parts |
| After M7 | 14 to 15 Oct | Real data, larger build, SD card, console | 63 real scenes lossless; 1024-pixel build tested on the board |

## 5.1 The start: from an image project to a satellite project

The project started from a smaller FPGA project, a Sobel edge detector on a Basys 3 board. The team examined that project, corrected its errors and made all its tests pass. Then the engineer asked for a larger project of the type that imaging satellites use. The team gave several ideas, and the engineer selected the payload data processor.

Before any hardware work, the team wrote the specification (SPEC.md). The specification gives the reference mission, the requirements of each block, the register map, the file formats, the tests and the milestones. The team also made an estimate of the work for each milestone.

## 5.2 M0: Tools and board start-up

### Objective

Make the tool flow and a first design that moves data between the PS memory and the PL.

### Work done

1. The team made a Git repository and the folder structure.
2. The team wrote Tcl scripts. The scripts make the Vivado project, the block design and the FPGA configuration with no manual steps.
3. The team made a block design with the PS, an AXI DMA controller and a FIFO in a loop.
4. The team wrote a test program for the Arm processor. It sends 64 MB through the loop and compares the data.

### Problems and solutions

| Problem | Cause | Solution |
|---|---|---|
| Data written at address 0x4000 also changed address 0x0. | The DDR4 memory module on the board (x16 devices) is different from the module in the Vivado board file (x8 devices). | The team read the data of the memory module (SPD) through I2C. Then the team changed the DDR settings in the design. |
| A program from the boot flash started and changed the memory during the test. | The board starts from the QSPI flash at each reset. | The test script sets the boot mode register to JTAG before the reset. This setting stays only until power-off. The flash does not change. |
| Vitis did not accept the project files. | The name of the project folder has a space. | The software workspace has a path with no spaces, through a directory link (junction). |

### Result

64 MB of data went through the loop with no error, at 3865 MB/s. This is 97 % of the theoretical rate.

## 5.3 M1: Python models

### Objective

Make a bit-exact software model of each block.

### Work done

The team wrote models for these functions:

- The CCSDS 123 compressor and the decompressor.
- The radiometric correction.
- Space packets, AOS frames and the functions that take them apart again.
- The Reed-Solomon encoder and decoder.
- The randomizer, the sync markers, the frame synchronizer and the error injection.
- The full chain, from raw scenes to the ground and back.

### Tests

Each model has automatic tests (pytest). The decompressor must give back the exact input for each compressed image. The Reed-Solomon encoder must give the same check bytes as an independent library (reedsolo).

### Result

138 tests passed at M1. At the end of the project, there are 150 tests.

## 5.4 M2: The compressor in hardware

### Objective

Make the CCSDS 123 compressor in the PL. It must be bit-exact with the model and operate at 200 MHz or more.

### Design decisions

- The compressor processes the samples pixel by pixel (BIP order). In this order, the data of one band is necessary again only after all the other bands. This gives time for a long pipeline, so the compressor processes one sample in each clock cycle.
- The previous image line stays in UltraRAM. Four 16-bit samples go into one 64-bit memory word. This decreased the number of UltraRAM blocks from 32 to 8.

### Tests

The tests use 28 compressor configurations, with two scenes each. The testbench compares each output byte with the model. It also compares each internal residual, so a difference shows the exact sample. The input has random gaps and the output has random stops.

### Result

All 28 configurations are bit-exact. The timing is correct at 250 MHz. The compressor uses 13 600 LUTs, 19 DSP blocks and 8 UltraRAM blocks.

## 5.5 M3: The sensor front end

### Objective

Make the sensor emulator, the radiometric correction and the corner turn.

### Work done

The team wrote the three blocks and one testbench that connects them to the compressor. The testbench uses two clocks with no fixed phase relation: 100 MHz for the sensor and 250 MHz for the data path.

### Problems and solutions

| Problem | Cause | Solution |
|---|---|---|
| A dropped line can make the compressor lose the scene boundaries. | All the lines after the dropped line move by one line. | The corner turn sends a line of zeros (a fill line) in place of a dropped line. A counter records each dropped line. |
| The timing margin was only 0.112 ns. | The UltraRAM read path went through a chain of 7 memory blocks with only one register. | The team added two more registers in the read path. The margin increased to 0.651 ns. |

### Result

12 tests are bit-exact through the compressor. The timing is correct at 250 MHz.

## 5.6 M4: First compression on the board

### Objective

Compress scenes on the board and decode the result on the PC.

### Work done

1. The team wrote the register map in SystemRDL and made the register logic with PeakRDL.
2. The team connected the register clock (100 MHz) to the data path clocks. Synchronized snapshots move the settings and the counters between the clocks.
3. The team added the front end and the compressor to the block design.
4. The team wrote the control software (hpdp_run) and a PC script (run_m4.py).
5. The team did a system simulation that operates the design through its registers, as the software does.

### Problem and solution

| Problem | Cause | Solution |
|---|---|---|
| The first part of the calibration table went to the wrong block. | The DMA started before the new route setting arrived in the data path clock. The setting arrives approximately 1 µs after the write. | A new status bit (cfg_pending) shows when all the register writes have arrived. The software waits for this bit. |

### Result

The system simulation found this problem before the board test. On the board, 15 scenes in 6 runs were bit-exact after decompression. The rate was 99.8 Msamples/s. This is the maximum rate of the sensor emulator.

## 5.7 M5: Downlink chain and optical link

### Objective

Send the compressed scenes through the full channel chain and the optical fiber, and get them back on the ground.

### Work done

- The packetizer, the AOS framer, the Reed-Solomon encoder, the randomizer and the error injector.
- The frame synchronizer for the ground.
- The link layer with its lock state machine.
- The GTH transceiver settings. These settings came from an earlier SFP link project of the engineer (JSB MARK 1), which operated on the same board.
- A new top level with the transceivers, the reference clock and the free-running clock.

### Tests on the board

- PRBS-31 test over the fiber for 45 minutes: $3.375 \times 10^{12}$ bits with no error. This shows a BER of less than $8.9 \times 10^{-13}$ at 95 % confidence.
- Scenes over the fiber with errors added on purpose: all scenes were bit-exact at BER 0, $10^{-4}$ and $10^{-3}$.
- Ten minutes at the mission rate (76.8 Msamples/s): no line was lost.

### Problem and solution

In the RAW test mode, the data has no compression. The data rate is then more than the link capacity, so lines are lost, as expected. But a scene with lost lines did not end, and the next scenes became part of it. The team added the scene guard (Section 3.2, item 4). Now each scene ends correctly.

## 5.8 M6: The PC ground station

### Objective

Decode, examine and show the ground data on the PC.

### Work done

The ground station (hpdp_gs.py) reads the records from a file or from a TCP stream. It corrects the errors, takes out the packets and the scenes, and decompresses each scene. It compares each scene with the reference. It shows false-colour images and statistics.

> **NOTE**
> The board also sends the records to the PC through Ethernet (requirement SW-05). The team did the test of this function on the board on 23 October 2026. The test found two faults in the board program and one fault in the PC program. The team corrected the three faults. After that, the records that came through Ethernet were the same as the records that came through JTAG.

## 5.9 M7: Radiation mitigation

### Objective

Find and correct the effects of radiation in the FPGA. Radiation can change single bits in memories and registers (single event upsets).

### Mechanisms

| Part | Protection |
|---|---|
| Configuration memory | The SEM controller reads the configuration memory continuously and corrects changed bits. |
| UltraRAM | Built-in ECC corrects one wrong bit and finds two wrong bits in a word. |
| Compressor memories, registers and state machines | ECC on block RAM. Three copies with a vote (TMR) for small memories, registers and state machines. |
| Pipeline and software | Watchdog timers in the PL and in the PS find a stalled part and stop or reset it. |
| Data with errors | The packet header marks the scene, and an interrupt tells the software. |

### Fault campaign

The team changed bits on purpose in a simulation, 1300 times. In the protected parts, no change caused a silent error. The campaign found unprotected registers in three blocks. The team added TMR to these registers in the final build (m7h).

### Tests on the board

The team changed 251 bits of the configuration memory on the board. The SEM controller corrected 198 of them in 125 ms to 153 ms. The other 53 bits were in unused areas. The scenes after each change were bit-exact.

## 5.10 After M7

- **Real data.** 63 public hyperspectral images (AVIRIS, Hyperion and Indian Pines) went through the compressor. All 63 came back with no loss.
- **Larger images.** A build for 1024 pixels in each line (NX_MAX = 1024) operated correctly on the board.
- **SD card and console.** The software can now load scenes from an SD card. A UART console lets an operator start the jobs.
- **Full test matrix.** All 864 compressor configurations of the specification are bit-exact.
- **Transfer package.** Two archives let you move the project to another computer.
- **Project report.** The report gives the status of each requirement.

# 6 Verification

The team tested the HPDP at five levels. Table 3 gives the levels.

*Table 3: Verification levels*

| Level | What the test does | Quantity |
|---|---|---|
| Python models | Automatic tests of each model, for example compression and decompression of many images | 150 tests |
| RTL simulation | Self-checking testbenches with data from the models. The data flow stops at random times. | 87 tests in 9 minutes; the full matrix of 864 configurations in 57 minutes |
| System simulation | The full design, operated through its registers as the software does | Part of the 87 tests |
| Fault campaign | Bits changed on purpose in a simulation; each result is classified | 1300 trials |
| Board tests | Runs through JTAG; the board output is compared with the model | Each milestone from M0 to M7, and after M7 |

A test passes only when every output bit agrees with the model. A result that is almost correct is a failure.

# 7 Procedures

This section tells you how to build, test and operate the HPDP. Do the procedures in the given sequence.

## 7.1 Prepare the PC

1. Install AMD Vivado and Vitis 2025.2 with support for Zynq UltraScale+ devices.
2. Install Python 3.10 or a later version.
3. Extract the transfer package (HPDP_2026-10-15.zip) to a folder on the PC.
4. Install the Python packages. Type: `pip install -r requirements.txt`
5. Make the directory links (junctions) that docs/new_system_setup.md gives.

> **NOTE**
> Vitis does not operate correctly when the path has a space or is very long. The junctions give short paths with no spaces.

## 7.2 Do the model tests

1. Open a command window in the project folder.
2. Type: `python -m pytest model`
3. Make sure that all the tests pass.

## 7.3 Do the RTL simulation tests

> **CAUTION**
> Do not change the RTL files while a Vivado build or a simulation runs. The tool can read a file while you change it, and the result is then not valid.

1. Type: `vivado -mode batch -source tb/run_sim.tcl`
2. Wait approximately 9 minutes.
3. Make sure that the summary shows “All tests passed”.

## 7.4 Build the FPGA configuration

> **CAUTION**
> If a build stops before the end, delete its build folder before you start again. Vivado does not start runs that an earlier build left open.

1. Set the environment variable HPDP_BUILD_DIR to a new folder, for example build/m7h.
2. Type: `vivado -mode batch -source vivado/build.tcl`
3. Wait until the build is complete. This takes approximately 30 to 60 minutes.
4. Make sure that the last line shows a WNS of 0 ns or more.
5. Make sure that the build folder contains the file hpdp.xsa.

> **NOTE**
> For the build with 1024 pixels in each line, also set HPDP_NX_MAX to 1024.

## 7.5 Build the software

1. Type: `vitis -s sw/baremetal/build.py build/m7h/hpdp.xsa`
2. Make sure that the file hpdp_run.elf is in the vitis folder.

## 7.6 Connect the board

**WARNING**
Do not look into the end of an optical fiber or into an SFP module when the board has power. The SFP modules contain lasers.

**CAUTION**
Do not install or remove an SFP module during a measurement. The change of supply current causes bit errors.

1. Make sure that the power switch of the board is off.
2. Install the two SFP modules in the SFP0 and SFP1 cages.
3. Connect the duplex OM3 fiber between SFP0 and SFP1.
4. Connect the USB JTAG cable to the PC.
5. Connect the USB UART cable to the PC.
6. Set the power switch to on.

**NOTE**
The boot mode switches can stay in the QSPI position. The test script sets the JTAG boot mode, and this setting stays only until the next power-off.

## 7.7 Do a board run

**CAUTION**
Use only JTAG to load the board. Do not write to the QSPI flash. The flash contains the start-up program of the board.

**CAUTION**
Do not stop Vivado or simulator processes that you did not start. Other users can run jobs on the same PC.

1. Type: `python sw/host/run_m5.py --work <name>`
2. Wait for the result. The script loads the FPGA configuration, the software and the scenes. Then it runs the job and reads the results back.
3. Make sure that the last line starts with PASS.
4. Find the files of the run in the folder build/runs/<name>.

## 7.8 Decode the data at the ground station

1. Type: `python sw/ground/hpdp_gs.py decode build/runs/<name>/gr.bin --ref build/runs/<name> --show`
2. Make sure that each scene shows “bit-exact”.
3. Look at the false-colour images and the statistics.

## 7.9 Troubleshooting

| Symptom | Possible cause | Action |
|---|---|---|
| The run stops with the status ST_LINK. LNK.STATUS shows 0xF07 and many code errors. | The fiber is disconnected. | Examine the fiber connections. To test the board without the fiber, do the run again with the option --loopback pma. |
| PRBS errors occur only at the start of a run. | An SFP module was installed during the run. | Do the run again. Do not touch the modules during a measurement. |
| The build shows a WNS less than 0 ns. | The timing is not correct. | Examine the timing report in the build folder. Do not use this build. |
| Vitis shows “invalid os”. | The path has a space. | Use the junction paths (Section 7.1). |
| Vivado does not start the build runs. | An earlier build stopped before the end. | Delete the build folder and start the build again. |

# 8 Lessons learned

- A bit-exact model before the hardware found most of the errors in the simulation. Few errors went to the board.
- The system simulation through the registers found a problem between clock domains (cfg_pending) before the board test.
- A test on the board can find problems that a test of one block cannot find. The scene guard is an example (Section 5.7).
- A fault campaign shows which parts need protection. It caused the TMR on the control registers in build m7h.
- Settings from an earlier project that operated (JSB MARK 1) made the transceivers operate at the first attempt.
- Each tool has limits, for example paths with spaces and builds that stop. Write down a limit when you find it.
- A separate result block for each job made each board run repeatable and self-checking.

# 9 Status

The HPDP meets 100 of the 102 requirements of version 1.0 of the specification. No requirement has the status “not met”. Two requirements are open:

- **CMP-02 and MDL-02** (comparison with an independent CCSDS 123 decoder) are open. They need an account for the ESA or the CNES tools.

The results report (HPDP-RPT-001) gives the results, the status of each requirement and the open items.

# 10 Technical names and abbreviations

| Name | Meaning |
|---|---|
| AOS | Advanced Orbiting Systems. The CCSDS standard for transfer frames. |
| ASM | Attached sync marker. The four bytes 1A CF FC 1D before each codeblock. |
| BER | Bit error rate. The number of wrong bits divided by the number of all bits. |
| BIL, BIP | Band interleaved by line, band interleaved by pixel. Two sequences of the samples of an image. |
| Bit-exact | The same bits as the reference, with no difference. |
| CADU | Channel access data unit. An ASM and a codeblock: 1024 bytes. |
| CCSDS | Consultative Committee for Space Data Systems. It writes the standards for space data. |
| Clock domain crossing (CDC) | A circuit that moves data safely from one clock to another clock. |
| Codeblock | A frame and its Reed-Solomon check bytes: 1020 bytes. |
| DMA | Direct memory access. A controller that moves data without the processor. |
| ECC | Error correction code. |
| FPGA | Field-programmable gate array. A device with logic that you can configure. |
| Gbaud | $10^9$ symbols each second on the line. |
| GTH | The high-speed serial transceiver of the XCZU7EV. |
| JTAG | The debug interface that loads the FPGA configuration and the software. |
| LUT, BRAM, URAM, DSP | FPGA resources: look-up tables, block RAM, UltraRAM and signal processing blocks. |
| Msamples/s | $10^6$ samples each second. |
| PL, PS | Programmable logic, processing system. The two parts of the device. |
| PRBS-31 | A pseudo-random bit sequence for the test of a link. |
| Pushbroom camera | A camera that records one line across the ground track at a time. The movement of the satellite gives the next line. |
| Reed-Solomon (255,223) | An error correction code. It adds 32 check bytes to 223 data bytes and can correct 16 wrong bytes. |
| RTL | Register transfer level. The description of the hardware in SystemVerilog. |
| SEM | Soft error mitigation. The AMD controller that corrects the configuration memory. |
| SFP | Small form-factor pluggable. An optical transmitter and receiver module. |
| STE | ASD-STE100 Simplified Technical English. The language of this document. |
| SystemRDL | A language that describes registers. |
| TMR | Triple modular redundancy. Three copies of a circuit and a vote. |
| WNS | Worst negative slack. The timing margin of the slowest path. It must be 0 ns or more. |